



Universidad de Jaén

Tema 4.1 Programación en el Cliente: El Lenguaje Javascript

Desarrollo de Aplicaciones Web

José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

v 1.4

Este obra está bajo una [Licencia Creative Commons](http://creativecommons.org/licenses/by-sa/4.0/)
Atribución-CompartirIgual 4.0 Internacional.

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Objetivos

- Recordar la **sintaxis** básica del lenguaje
- Profundizar en el manejo de **tipos de datos** primitivos y abstracciones predefinidas del lenguaje
- Conocer las características especiales y uso de las **funciones** en Javascript
- Comprender y saber utilizar aspectos fundamentales del modelo de **objetos del lenguaje**
- Conocer los aspectos básicos de las **Expresiones Regulares** y saber utilizarlas como mecanismo de validación de datos en aplicaciones web



El lenguaje Javascript

- Aparece en 1996 (*Brendan Eich*) en navegador Netscape
- Estandarizado como ECMAScript [última versión](#) (e.g. ES2025, versión ES16)
 - **ES5-6** más soportado actualmente por navegadores. Verificar soporte en caniuse.com
- **Alternativas**
 - "*Polyfills*", módulos JS para emular funcionalidades de JS o de navegadores de versiones superiores
 - "*Transpiladores*" ES678 → ES5, e.g. Babel
 - "*Meta-lenguajes*" (traducen a JS): Typescript, Coffescript, Dart...
- **Características generales**
 - Interpretado
 - Tipado dinámico
 - Sintaxis C, Java y lenguajes funcionales
- **Usos:** programación en cliente y/o servidor (e.g. NodeJS, DenoJS, Nashorn,...)

3

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Características sintácticas

- Diferencia entre mayúsculas y minúsculas
- Fin de línea con ; opcional (recomendado!)
- Utilizar \ para dividir en varias líneas
- **Comentarios:** */**/* y *//*
- **Variables:**
 - El tipo se asocia al valor, no a la variable (**tipado dinámico**)
 - **Declaración:**
 - `var variable=valor;` //no omitir var, let o const!
 - `let variable=valor;`
 - `const IVA=0.26;` //No puede modificarse
 - **Ámbito:**
 - Local: dentro de una función y derivados (parámetros o var)
 - Local estricto: Solo en bloque donde está definida { } (let)
 - Global: fuera de una función o sin usar var (!)

Se recomienda utilizar **'use strict'**; en ficheros o funciones para evitar construcciones que puedan generar errores potenciales, e.g. variables globales por no usar var

4

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Tipos de valores reconocidos

Tipos primitivos

- Numérico(number) enteros y reales, *NaN*, *Infinity*
- Lógico(boolean) *true* | *false*
- Cadenas(string) "Hola", 'Hola \n "Campeón" '
 `Hola \${usuario}` //sustituir variable
 `Texto` // Cadena multilinea
 muuuuuy
 largo`
- *undefined* valor no inicializado

Ejemplos

- `let temperatura=25.6;`
- `let encontrado=true;`
- `let nombre="Paco"+" Martínez"`
- `let aux;` //valor undefined
- `const iva=0.26;`

5

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Operadores

• Similares a Java

■ Aritméticos:

- binarios: + (suma y concatenación), -, *, **, /, %
- unarios: ++, --

■ Lógicos: &&, ||, ++, ! (negación)

■ Relacionales:

- <, >, <=, >=,
- === comparación sin conversión de tipo (!==)
- == comparación con conversión implícita (!=)
 - Ejemplos:
 - `3=="3"` true!
 - `3===3"` false

■ typeof devuelve el tipo de un valor

- `typeof nombre;` // string
- `typeof aux;` //undefined

6

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Estructuras de control condicional

```
if (condición) {  
    ...  
} else {  
    ...  
}
```

Importante:

valor **falso**: false, null, undefined, "", 0, NaN
valor **verdadero**: cualquier otra cosa ("false!")

```
switch (valor) {  
    case valor_1:  
        ...  
        break;  
    case valor_2:  
        ...  
        break;  
    default:  
        ...  
}
```

Estructuras de control cíclicas

```
for(let i=0;i<valor;i++){  
    ...  
}  
for(let key in colección) {  
    ...  
}  
for(let value of colección){  
    ...  
}
```

```
while (condición) {  
    ...  
}  
do {  
    ...  
} while (condición);
```

Se puede hacer (buen) uso de **continue** (forzar iteración) y **break** (finalizar iteración)

Excepciones

Sintaxis:

```
try {
    throw expresión
} catch (excepción) {    //opcional
    //tratamiento de excepción
} finally {              //opcional
    //Ejecución tras try o catch
}
```

Ejemplo:

```
let i=0;
try {
    for (i=0; i<v.length()-1;i++) {
        if (v[i]==0)
            throw new Error("div By 0");
        v[i+1]=v[i];
    };
} catch (e) {
    console.log("Error["+e.name+"]: "+e.message);
} finally {
    console.log(i+" elementos procesados");
}
```

Algunos objetos excepción de Javascript pueden reutilizarse: *Error*, *EvalError*, *InternalError*, *RangeError*, *ReferenceError*, *TypeError*, *URIError*

Las excepciones que no se capturan se propagan al nivel superior hasta que son capturadas o termina la ejecución

Sintaxis básica de objetos

- Implementados como *vectores asociativos*:
 - Acceso a un elemento por una clave alfanumérica
- **Declaración:**
 - `let obj={clave1:valor1, clave2:valor2,...};`
 - Ejemplo:
 - `let personal ={nombre:"Paco", edad:23};`
 - `let persona2 = null;` //objeto no creado
- **Acceso:**
 - Sintaxis objeto: `personal.nombre;`
 - Sintaxis de Arrays: `personal["edad"]=34;`
- **La asignación es una referencia!**
 - `let persona2 = personal;` //son la misma persona
 - `persona2.edad++;` //ahora personal.edad==24
- **Todos los objetos son del tipo object**
 - `typeof personal;` //object

Tipos de Objetos

- El lenguaje dispone de tipos de objetos asociados a cada tipo primitivo:
 - **Number, String, Boolean**
- Estos objetos disponen de métodos con funcionalidades habituales:

```
let nombre="Carlos";           //tipo string
let valor1=25.123;             //tipo number
let valor3=valor1.toFixed(2);  //25.12
let lonNombre=nombre.length;   //6
let cadena=valor1.toString();  //"25.123"
```

Según el contexto, Javascript transforma los tipos primitivos a objetos de su tipo asociado

11

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Strings

- Son elementos inmutables
- **Algunos métodos** (y una propiedad)

- **length**
- **concat(cad)** Concatenación (similar a +)
- **toLowerCase(), toUpperCase()**
- **charAt(index)** (Índices en 0. También con [index] solo lectura)
- **indexOf(subcad)** Posición de subcad o -1
- **substring(ini [,fin]), substr(ini,numelem)**
- **split(separador)** Array con elementos separados por carácter


```
var cadena="Luis;Gil;23";
var datos=cadena.split(";");//["Luis","Gil","23"];
```
- **trim()** Elimina espacios iniciales y finales

- **Funciones útiles**

- **parseInt(cadena [, base])**
Valor numérico o NaN
- **parseFloat(cadena)**
Valor numérico o NaN

12

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Arrays

Declaración

```
let notas      = [3.5, 6, 5.6, 8];  
let valores    = new Array(10); //undefined values  
let resp      = new Array("Sí", "No", "ns/nc");  
let matriz     = [[2,4],[5,6]];
```

Acceso

- índices numéricos (comienzan en 0)
- Sintaxis de corchetes:

```
notas[2]=25; matriz[0][0]=34;
```

Contenido heterogéneo: `notas[1]="suspenseo";`

Tamaño variable: `notas[5]=9; //notas[4] undefined`
`notas.push(10);`

13

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025

Uso de Arrays

- Los Arrays son objetos de tipo *Array*

- **Algunos métodos:**

- **length** Longitud del array
- **push(),pop()** Añadir/Extraer al final
- **shift(),unshift()** Añadir/extraer principio
- **sort()** Ordenar un array
- **reverse()** Invertir posición elementos
- **join("separador")**
 - Devuelve una cadena con elementos unidos por separador
- **concat(array)** Array con la concatenación de otros 2
- **slice(ini,fin)** Obtiene un subvector de elementos
- **splice(ini,num[,newEl1,newEl2,...])**
 - Borra num elementos desde ini y, opcionalmente añade newEl1, ...
- **indexOf(elem,ini)** Busca elem y devuelve su pos. o -1
- **includes(elem)** Busca elem y devuelve *bool*
- **toString()** Cadena con elementos separados por ,

14

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025

Iteración sobre arrays

El bucle for..in devuelve índices, no valores

- **forEach(function(val, index, array))**

- Ejecuta una función para cada índice, valor

```

let dias=["lun","mar","mie","jue","vie","sab","dom"];
dias.forEach( function (val, pos) {
  console.log( `${pos} : ${val}` );
})
for (let dia of dias) {
  console.log( dia );
}

```

- **every(fn), some(fn):**

- Igual que forEach pero permiten terminar la iteración cuando la función devuelve false o true respectivamente

```

dias.every ( function (val, pos) {
  if (val=="sab") return false;
  console.log(pos+": "+val);
  return true;
})
//muestra 0:"lun" a 4:"vie"

```

15

 Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025


Manipulación de Arrays

- métodos especializados para manipular todos los elementos del array

```

let productos=[ {nombre:"tuercas",precio:6 },
  {nombre:"tornillos",precio: 9},
  {nombre:"puntas", precio: 2}];

//devolver copia con valores procesados
let nombres=productos.map(function(p) {return p.nombre.toUpperCase() });
//['TUERCAS','TORNILLOS','PUNTAS']

//devolver copia con elementos seleccionados
let filtrados=productos.filter( function (p) { return p.precio>5 } );
//[ {nombre:'tuercas',...}, {nombre:'tornillos',...} ]

//devolver un valor resultado de procesar todos los elementos
let total=productos.reduce( function (acum,p) {return acum+p.precio}, 0/*valor inicial*/); //17

//... y pueden combinarse
let nombres=productos.filter( function (p) { return p.precio>5 } )
  .map( function(p) {return p.nombre.toUpperCase() }); //['TUERCAS','TORNILLOS']

```

16

 Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025


Estructuras Map y Set (ES6+)

```

var usuarios = new Map();
usuarios.set( "pepe" , {name:"Pepe", age: 23} );
usuarios.set( "maria" , {name:"Maria", age: 20} );
console.log( usuarios.size );
console.log( usuarios.get("pepe").age );
usuarios.delete("pepe");
for (let [ key , value ] of usuarios) {
    console.log( `${key} tiene ${value.age} años` );
}
usuarios.forEach( function (value, key) {
    console.log( `${key} tiene ${value.age}; años` );
});

var meses = new Set( ["enero",
                      "febrero","marzo"] );
meses.add( "abril" );
console.log( meses.size );
meses.delete("marzo");
console.log( meses.has("marzo") ); //false
for (let m of meses) { console.log( m ); }
meses.forEach( function (value){
    console.log(value) ;
});
  
```

17

 Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025


Funciones

● Sintaxis:

```

function f1 (p1, p2 ="defecto" ,...) {
    let vlocal1; //p1, p2, vlocal1: locales
    vlocal2; //global!
    ...
    return vlocal1;
}
let fn= p1 => expr o (p1,p2) => {...} //Arrow functions ES6+
fn(valor);
  
```

- **Parámetros:** simples, por copia; objetos, por referencia!
- Si se pasan menos parámetros actuales → undefined
- Si se pasan más (o no se declaran parámetros formales)
 - Acceso mediante objeto **arguments**
 - `arguments.length;` //número de parámetros
 - `arguments[0];` //primer parámetro

18

 Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025


Las funciones son objetos!

Se pueden asignar a variables y pasar como parámetros...

```
function aplica(v, func) {
  for (let i=0; i<v.length;i++) {
    v[i]=func(v[i]);
  }
}

let cuadrado = function (valor) {      //función anónima o Lambda
  return valor*valor;
}

let incrementa = valor => valor+1      ;//funciones anónimas con =>
let max = (val1,val2) => { return val2>val1?val2:val1; };

let valores=[1,2,3,4,5];
typeof cuadrado;                      //function
cuadrado(4);                          //16
incrementa(5);
max(5,6);
aplica(valores,cuadrado);             // valores=[1,4,9,16,25]
(function(x,y){return x+y;})(4,5);    //9. Definir y ejecutar:
//Immediately-invoked Function Expressions (IIFE)
```

19

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Modelo de Objetos

- **Arrays asociativos:** colecciones no ordenadas de pares (cadena,valor), donde *valor* puede ser un tipo simple o un objeto (*las funciones son objetos!*)

- **Creación:**

- **Literal:** `let a={}, b={nombre:"Juan",edad:23};`

- Operador **new** y función Constructora:

```
let o = new Object();                //similar a let o={};
function Persona(nombre) {           //en Mayúsculas por convención
  this.nombre=nombre;               //this: referencia al objeto creado
}
let p1 = new Persona("Paco");
let personas = new Array ( p1, p2, new Persona("Maria") );
typeof p1;                          // object ?!
p1 instanceof Persona               // true
```

- **Object.create(prototipo) (>IE7)**

```
let o= Object.create(null);          //similar a let o={};
```

20

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Modelo de Objetos

- Los objetos **son dinámicos**: pueden añadir/borrar propiedades... de cada objeto!

```
let p1=new Persona("Juan");
let p2=new Persona("María");
p1.apellidos="González";
p2.apellidos;           //undefined!, p2 no tiene esa propiedad
p1["nombre"];           //acceso como array asociativo-> Juan
delete p1.nombre;       //eliminar propiedad
```

- El bucle **for..in** itera sobre propiedades (no valores)

```
for(let prop in p1) {
    console.log(prop+": "+p1[prop]+"\\n");
}
```

21

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025

Herencia orientada a prototipos

- Los objetos creados heredan las propiedades de otro objeto conocido como su "**prototype**"
 - Objetos literales, heredan del prototipo del objeto global.
 - Creados con new, heredan del prototipo de la función constructora... y se puede cambiar!
- Si se añade una propiedad a un prototipo la "heredan" todos los objetos creados a partir de él
- Si se modifica una propiedad heredada se crea una copia local

```
function Persona(nombre) { this.nombre=nombre; };
let p1= new Persona("Paco");
let p2= new Persona("María");
p2.edad=27;           //Cambio Local del objeto p2
Persona.prototype.apellidos="Desconocido";
p2.apellidos;         //"Desconocido"
```

22

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025

Definición de métodos de objetos

Los métodos deben asignarse al prototipo para que sean compartidos por todos los objetos

```
function Persona(nombre) {
  this.nombre=nombre;
  this.getNombre1= function () { return this.nombre; }
  //Función anónima: sin nombre. Una copia en cada objeto!
}

Persona.prototype.getNombre=function () {return this.nombre;}
Persona.prototype.setNombre=function (nom) {
  this.nombre = nom || "Sin nombre";
}

let p1= new Persona("Paco");
let p2= new Persona("María");
p1.getNombre();           //Paco
p2.setNombre();           //p2.nombre="Sin Nombre"
p2.edad=19;
```

Persona.prototype

getNombre()
setNombre()

getNombre1()
nombre: Paco

p1

getNombre1()
nombre: María
edad=19

p2

23

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Class (ES6+)

Realmente son construcciones sintácticas para emular los elementos tradicionales de lenguajes de POO con clases

```
class Persona {
  constructor (nombre, edad=18) {
    this._nombre=nombre;
    this._edad=edad;
  }
  //methods
  getEdad() {
    return this._edad;
  }
  //Properties
  get edad() {
    return this._edad;
  }
  set edad(edad) {
    if (edad<18) {
      throw new Error ("No se admiten menores de edad");
    }
    this._edad=edad;
  }
  //Class methods
  static crea (nombre) { return new Persona(nombre); }
};
```

```
class Estudiante extends Persona {
  constructor (nombre, edad, titulacion="Informática") {
    super(nombre,edad);
    this._titulacion=titulacion;
  }
  getTitulacion() {
    return this._titulacion;
  }
};

var e = new Estudiante("Pepe", 25);
console.log(`${e._nombre} ${e.edad} ${e.getTitulacion()}`);

var e1= Persona.crea("Carlos");
e1.edad=16; //Error exception!
```

24

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Expresiones Regulares

Introducción y uso con Javascript

Expresiones regulares

- Representadas mediante objetos **RegExp**
- Definen patrones de texto que pueden localizarse en cadenas
- Sintaxis: /patrón/opciones
- Los objetos de tipo *String* y *RegExp* tienen métodos para localizar y procesar patrones de texto
- Muy útiles en aplicaciones web para validación de datos en cliente y servidor. Disponibles en todos los lenguajes de programación
- Ejemplo

```
let cadena= "no te olvides del DOCTYPE";
let patron=/del/;           //equivalente a RegExp("del")
cadena.search(patron);      //Devuelve 14
cadena.search(/doc/);       //-1, No se encuentra
```

Expresiones Regulares

Las expresiones regulares utilizan notaciones especiales que definen conjuntos de caracteres

[...]	Cualquier carácter incluido. e.g. [abc],[a-z]
[^...]	Cualquier carácter no incluido. e.g. [^0-9]
.	Cualquier carácter excepto fin de línea
\w	Cualquier letra y _ ([a-zA-Z0-9_])
\W	Carácter que no es letra ([^a-zA-Z0-9_])
\s	Espacio en blanco
\S	Carácter que no sea un espacio
\d	dígito ([0-9])
\D	Carácter que no sea dígito ([^0-9])

En las ER en java, el carácter \ debe escaparse para no confundirse con un carácter especial. e.g. \\w

nota: para incluir un carácter especial como normal, utilizar \ como carácter de escape. e.g [\w] → \ o w; \. → localizar un punto

27

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Expresiones Regulares

Repetición de patrones:

?	Cero o una ocurrencias.	e.g. /\w?/
*	Cero o más ocurrencias.	e.g. /\w\s*\w/
+	Una o más ocurrencias.	e.g. /\d+/
{n}	Exactamente n ocurrencias.	e.g. /\d{3}/
{n,}	n o más veces	
{n,m}	Entre n y m veces.	e.g. /\d{7,8}/

Se pueden utilizar **paréntesis** para agrupar patrones

`/\d{7,8}(-[A-Z])?/`
`/\w+@ \w+(\. \w+)+/`

28

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Expresiones Regulares

Patrones alternativos separados por |

```
/a|b/, /\w+|\d+/, /(\d{3}|\w{4})?/  
/[Ss][ÍíIi]|[Nn][Oo]/
```

Posición del patrón en la cadena

^ Comienzo de la cadena/línea. `/^\d\w+/`

\$ Final de la cadena/línea `/^\d+€$/`

Opciones de búsqueda

Especificadas después del patrón

i no distingue may. y min. `/s[íi]|no/i`

g Busca todas las ocurrencias `/\d+/g`

29

 Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025


Uso de expresiones regulares

• Métodos del tipo String

- **search(regex)** posición donde aparece o -1
- **replace(regex, cadena)** reemplaza ocurrencias
- **match(regex)** Array con ocurrencias o null
- **split(regex)** Trocea usando la ER como separador

• Métodos de RegExp

- **exec(cadena)** Similar a String.match (permite iteración)
- **test(cadena)** true si se encuentra el patrón en cadena

```
let cadena="Mi dni es 11223344-T";  
cadena.search(/\d{7,8}(-?[a-z])?/i); //posición 10  
cadena.search(/^\d{7,8}(-?[a-z])?$/i); //-1  
cadena.replace(/dni/, "NIF"); //Mi NIF es ...  
let result=cadena.match(/\d{7,8}(-?[a-z])?/i); //result=["11223344-T", "-T"]  
/\d+/.test(cadena) //true
```

30

 Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025


Uso de Expresiones Regulares

Comprobar si un número de tarjeta es correcto y obtener sus cuatro números

```
var cad1="1234-5678-9012-3456";
var cad2="1234-5678-9012-345a";
var regExpCard=/^(\d{4})-(\d{4})-(\d{4})-(\d{4})$/;
var nums1=cad1.match(regExpCard);
var nums2=cad2.match(regExpCard);

//nums1==["1234-5678-9012-3456", "1234", "5678", "9012", "3456"]
//nums2== null. El último grupo de la tarjeta no es válido
```

nota: si se encuentra el patrón, la posición 0 del array muestra el patrón encontrado y el resto los grupos () definidos

Eliminar las etiquetas html de una cadena

```
var texto=cadena.replace(/<[^>+>/ig, "");
```

31

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Conclusiones

- **Javascript** o ECMAScript es un lenguaje estandarizado
- JS es un **lenguaje interpretado**, con sintaxis y características similares a C++, Java y lenguajes funcionales
- El **intérprete de JS en los navegadores** puede no soportar las características más recientes de JS aunque en actualidad el soporte de ES5 y ES6 es completo
- Las **expresiones regulares** son un mecanismo presente en diferentes lenguajes de programación que facilitan la manipulación de textos y, en particular, son útiles para la validación de datos en aplicaciones web

32

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Bibliografía

- Zakas, Nicholas C. Understanding ECMAScript 6 : the definitive guide for JavaScript developers. No Starch Press, 2016. Available at [[Recursos electrónicos](#)]

Lecturas Complementarias

- Javier Eguiluz. *Introducción a Javascript*. Uniwebsidad, 2006: Retrieved April 2025 from [[archive.org](#)]
- *Javascript Guide*. Mozilla Foundation. Retrieved March 2025 from <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
- Javascript API reference. W3schools, 2024. Retrieved March 2025 from <http://www.w3schools.com/jsref>
- Douglas Crockford. *The World's Most Misunderstood Programming Language Has Become the World's Most Popular Programming Language*. 2008. Retrieved April 2025 from <http://crockford.com/javascript/popular.html>